

Counting 4-Cycles in Fully Dynamic Graphs: A Cyclic-Join Perspective

Anonymous Author(s)

Abstract

[TODO: write abstract]

CCS Concepts

• **Theory of computation** → **Dynamic graph algorithms**; *Streaming, sublinear and near linear time algorithms*; • **Information systems** → **Database query processing**.

Keywords

fully dynamic algorithms, subgraph counting, 4-cycles, fast matrix multiplication, incremental view maintenance, cyclic joins

1 Introduction

1.1 Subgraph counting and dynamic graphs

[TODO: motivation paragraph]

1.2 The cyclic-join framing

[TODO: cyclic-join paragraph; insert Figure 1]

1.3 The state of the art

[TODO: ladder discussion + Table 1]

1.4 The Assadi–Shah result

[TODO: state Theorem 1 + the four-bullet surprise discussion]

1.5 What this essay does

[TODO: roadmap paragraph]

2 Preliminaries

2.1 Graphs and subgraph notation

[TODO: notation block]

2.2 Dynamic graph model

[TODO: define fully-dynamic update sequence; worst-case vs amortized]

2.3 Fast matrix multiplication

[TODO: omega definitions]

2.4 The OMv conjecture

[TODO: OMv paragraph]

3 The cyclic-join equivalence

3.1 Joins as layered graphs

[TODO: joins-as-layered-graphs paragraph; reuse Figure 1]

¹ $\varepsilon > 0$ is a constant depending on the matrix-multiplication exponent ω ; see Theorem 5.

FIG 1 PLACEHOLDER

Figure 1: A 4-layered graph encoding a cyclic join $A \bowtie B \bowtie C \bowtie D$. One layered 4-cycle highlighted. Adapted from [1], Fig. 1.

Pattern	Upper bound	Lower bound
3-cycle (triangle)	$O(\sqrt{m})$ [4]	$\Omega(m^{1/2-\gamma})$ [3]
4-cycle	$O(m^{2/3-\varepsilon})$ [1] ¹	$\Omega(m^{1/2-\gamma})$ [3]
4-clique	$O(m)$ folklore	$\Omega(m^{1-\gamma})$ [2]

Table 1: State of the art for fully dynamic subgraph counting. Lower bounds are conditional (OMv for 3- and 4-cycles; combinatorial 4-clique conjecture for 4-cliques).

FIG 2 PLACEHOLDER

Figure 2: Lifting a general graph G to a 4-layered graph G' : every vertex of V is replicated in each of the four layers, and an edge $(u, v) \in E$ becomes four lifted edges between consecutive layers.

3.2 Lifting a general graph to a 4-layered graph

[TODO: describe lifting; insert Figure 2]

3.3 Equivalence theorem

Theorem 1 (General-layered equivalence). *For the lifted graph G' defined above and operated under the update protocol of Section 3.2, the number of length-3 walks from $u \in L_1$ to $v \in L_4$ in G' equals the number of length-3 paths from u to v in G .*

PROOF. [TODO: proof body] $\square$

3.4 Consequence

[TODO: transition paragraph]

4 A simple $O(n)$ algorithm

4.1 The wedge-counting idea

[TODO: wedge identity paragraph]

Algorithm 1 Maintain 4-cycle count via wedge matrix W .

```

1: procedure INSERT( $u, v$ )
2:    $\Delta \leftarrow \sum_{w \in N(u)} W[w, v]$  ▷ query
3:    $C \leftarrow C + \Delta$ 
4:   for  $w \in N(u)$  do  $W[w, v] += 1$ ;  $W[v, w] += 1$ 
5:   end for
6:   for  $w \in N(v)$  do  $W[w, u] += 1$ ;  $W[u, w] += 1$ 
7:   end for
8:   add  $(u, v)$  to  $E$ 
9: end procedure
10: procedure DELETE( $u, v$ )
11:   remove  $(u, v)$  from  $E$ 
12:   for  $w \in N(u)$  do  $W[w, v] -= 1$ ;  $W[v, w] -= 1$ 
13:   end for
14:   for  $w \in N(v)$  do  $W[w, u] -= 1$ ;  $W[u, w] -= 1$ 
15:   end for
16:    $\Delta \leftarrow \sum_{w \in N(u)} W[w, v]$  ▷ query
17:    $C \leftarrow C - \Delta$ 
18: end procedure

```

4.2 The algorithm

[TODO: prose around Algorithm 1 explaining the order-of-operations subtlety]

4.3 Correctness and complexity

Lemma 2 (Correctness and complexity of Algorithm 1). *Algorithm 1 maintains the exact 4-cycle count of any fully dynamic graph in worst-case update time $O(n)$.*

PROOF. [TODO: proof body] □

4.4 Why $O(n)$ is not enough

[TODO: limitation paragraph]

5 The warm-up algorithm**5.1 Setup and assumptions**

[TODO: state assumptions; goal $O(m^{2/3-\epsilon_1})$]

5.2 Vertex degree classes

[TODO: define H/M/L on L_1, L_4 and S/D on L_2, L_3 per chunk]

5.3 Chunks and lazy maintenance

[TODO: describe chunked maintenance and lazy evaluation]

5.4 Data structures

[TODO: walk through the table; gloss the database-language interpretation]

Lemma 3 (Update-time accounting for Table 2; [1]). *The worst-case time to update all data structures in Table 2 is $O(m^{2/3-\epsilon_1})$ per update, with $O(m^{4/3-2\epsilon_1})$ amortized over chunk transitions.*

5.5 Worked example: the FMM step

Theorem 4 (Cost of the rectangular FMM step). *The matrix product $A^{L*} \cdot B_{i,DD}$ can be computed in time $m^{\omega(2/3+2\epsilon_1/3-\epsilon_1+\epsilon_2, 1/3-\epsilon_1+\epsilon_2)}$.*

Class	Stored products
High in L_1, L_4	$A^{H*} \cdot B_{<i}; A^{H*} \cdot B_{<i} \cdot C^{*H}; B_{<i} \cdot C^{*H}$
Medium in L_1, L_4	$A^{M*} \cdot B_{<i}; B_{<i} \cdot C^{*M}$
Low in L_1, L_4	$A^{L*} \cdot B_{<i,DD}; A^{L*} \cdot B_{<i,SS}; A^{L*} \cdot B_{<i,SD},$ and three symmetric products with C^{*L}

Table 2: Data structures maintained by the warm-up algorithm. Adapted from [1], Table 1.

Dataset	n	m	Simple-Wedge $\bar{t}$	Recompute $\bar{t}$
ca-GrQc	TBD	TBD	TBD	TBD
email-Enron	TBD	TBD	TBD	TBD

Table 3: Real-world graph experiments. $\bar{t}$ in milliseconds, median over 1000 updates.

PROOF. [TODO: dimension count + invocation of rectangular FMM] □

[TODO: discussion: this constraint (paper Eq (5)) is what forces $\epsilon_1 > 0$, with the system feasible at the numeric values stated in paper §3.4.]

5.6 Pulling it together

[TODO: summarize the query case analysis (HH, HM/ML/HL/MM, LL); cite Lemma 3.8]

5.7 From warm-up to the full theorem (sketch)

[TODO: one paragraph: aggregation breaks once A updates; the paper introduces phases of size $m^{1-\delta}$ that allow FMM precomputation on the (now-frozen) previous phase. Constraint Eq (9) of [1] forces $\omega < 5/2 - O(\epsilon)$, which is why Strassen ($\omega \approx 2.81$) is not sufficient. State the headline result we do *not* re-prove:]

Theorem 5 (Main result of [1], restated). *There is an algorithm for counting 4-cycles in any fully dynamic graph in worst-case update time $O(m^{2/3-\epsilon})$ for some constant $\epsilon > 0$. With current $\omega = 2.371339$, $\epsilon \approx 0.0098$; with $\omega = 2$, $\epsilon = 1/24$.*

[TODO: closing note: “We restate Theorem 5 verbatim from [1]; its proof requires combining the warm-up of Section 5, the phases machinery sketched above, and removal of the simplifying assumptions (paper Sections 6–7), which is beyond our scope.”]

6 Experiments**6.1 Setup**

[TODO: implementation details, hardware, datasets (synthetic ER, BA; one SNAP graph)]

6.2 Methodology

[TODO: methodology paragraph]

6.3 Results

[TODO: discussion of patterns: dense vs sparse, crossover position]

FIG 3 PLACEHOLDER

Figure 3: Median per-update time vs. n (log-log) for Simple-Wedge and Naive-Recompute at three densities.

FIG 4 PLACEHOLDER

Figure 4: Cumulative time vs. number of updates: crossover where dynamic maintenance overtakes recomputation.

6.4 Connection to the theoretical bounds

[TODO: honest caveat + observations connecting back to Section 4]

7 Conclusion

7.1 What we did

[TODO: one-paragraph recap]

7.2 Open questions

[TODO: open questions paragraph]

References

- [1] Sepehr Assadi and Vihan Shah. 2025. An Improved Fully Dynamic Algorithm for Counting 4-Cycles in General Graphs using Fast Matrix Multiplication. arXiv:2504.10748 [cs.DS] PODS 2025, Article 091. <https://doi.org/10.1145/3725228>.
- [2] Kathrin Hanauer, Monika Henzinger, and Qi Cheng Hua. 2022. Fully Dynamic Four-Vertex Subgraph Counting. In *1st Symposium on Algorithmic Foundations of Dynamic Networks (SAND 2022) (LIPIcs, Vol. 221)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 18:1–18:17.
- [3] Monika Henzinger, Sebastian Krininger, Danupon Nanongkai, and Thatchaphol Saranurak. 2015. Unifying and Strengthening Hardness for Dynamic Problems via the Online Matrix-Vector Multiplication Conjecture. In *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing (STOC)*. 21–30.
- [4] Ahmet Kara, Hung Q. Ngo, Milos Nikolic, Dan Olteanu, and Haozhe Zhang. 2020. Maintaining Triangle Queries under Updates. *ACM Transactions on Database Systems* 45, 3 (2020), 11:1–11:46.